Deep Learning and OCT Imaging: A Novel Ensemble Approach for Eye Disease Diagnosis – Proof Document

Software and Hardware used:

Software:

- Python 3.12
- CUDA 12.3
- Imageio 2.35.1
- Joblib 1.4.2
- Keras 3.5.0
- Matplotlib 3.9.2
- Numpy 1.26.4
- OpenCV-Python 4.10.0.84
- Pandas 2.2.2
- Scikit-learn 1.5.1
- TensorFlow 2.17.0

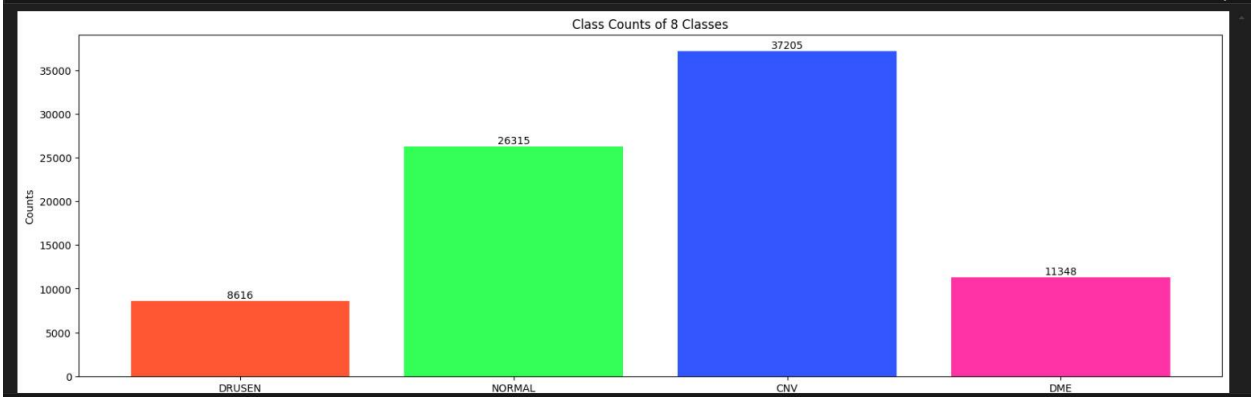
Hardware:

- Intel 13th Gen Intel(R) Core(TM) i7-13620H 2.40 GHz
- Nvidia Geforce RTX 4060

Code Snippets:

```python
data_path = 'OCT/train'
classes = os.listdir(data_path) # names of folders are the names of classes
class_counts = [len(os.listdir(data_path + '/' + x)) for x in classes] # length of respective classes is the count of images available for the respective class
#print(class_counts)
plt.figure(figsize=(20, 6))
bars = plt.bar(classes, class_counts, color = ['#FF5733', '#33FF57', '#3357FF', '#FF33A6', '#FF8633']) # 5 random colors chosen for better visualisation
plt.xlabel('Classes')
plt.ylabel('Counts')
plt.title('Class Counts of 8 Classes')
# Code to display count of each class on top of respective bar
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height,
        f'{height}',
        ha='center',
        va='bottom'
    )
```

Python

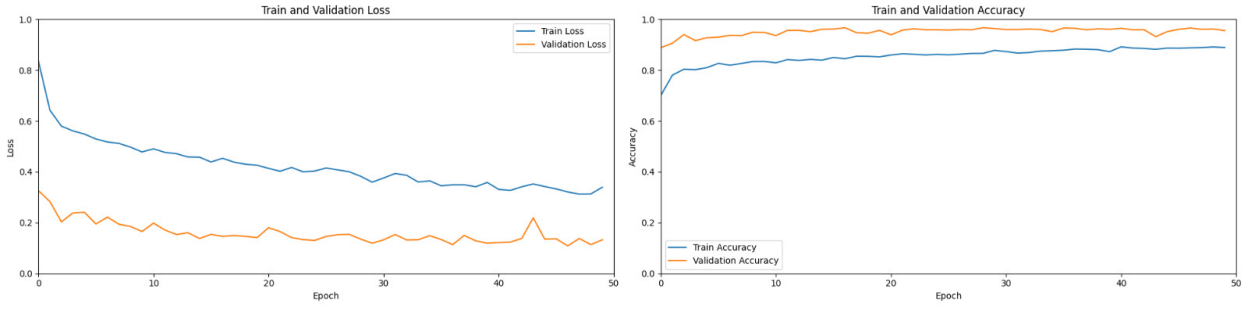


Distribution of training images

```python
base_model_0 = InceptionResNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet',
    pooling='max'
)

for layer in base_model_0.layers:
    layer.trainable = False

# Build model
inputs = base_model_0.input
x = BatchNormalization()(base_model_0.output)
x = Dense(1024, activation='relu')(x)
x = Dense(512,
x = Dense(512,  (function) activation: Any
x = Dense(256, activation='relu')(x)
x = Flatten()(x)
outputs = Dense(4, activation='softmax')(x)
model_0 = Model(inputs=inputs, outputs=outputs)

# Compile model
model_0.compile(optimizer=Adam(learning_rate=0.001), loss='categorical_crossentropy', metrics=['accuracy'])

# Deal with class imbalances
class_weights = compute_class_weight(class_weight='balanced', classes = np.unique(train_image_generator.classes), y= train_image_generator.classes)
class_weight_dict = dict(enumerate(class_weights))
```

Python

InceptionResNetv2 model

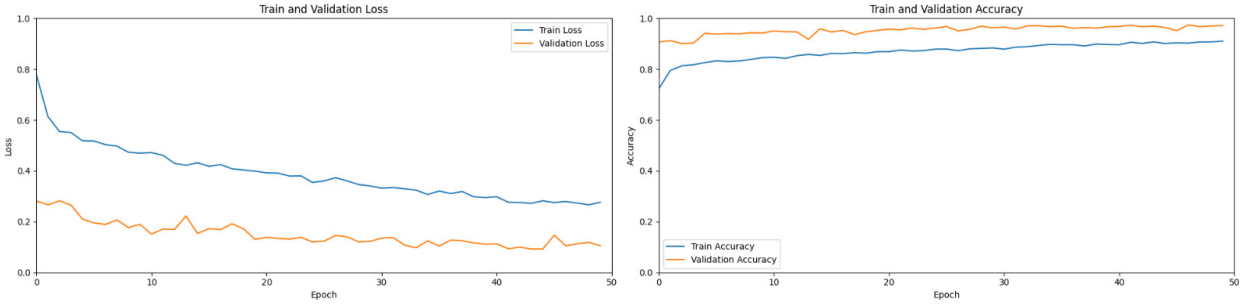


Inception ResNetv2 training and validation data

```python
base_model_1 = InceptionV3(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet',
    pooling='max'
)

for layer in base_model_1.layers:
    layer.trainable = False

# Build model
inputs = base_model_1.input
x = BatchNormalization()(base_model_1.output)
x = Dense(1024, activation='relu')(x)
x = Dense(512, activation='relu')(x)
x = Dense(512, activation='relu')(x)
x = Dense(256, activation='relu')(x)
x = Flatten()(x)
outputs = Dense(4, activation='softmax')(x)
model_1 = Model(inputs=inputs, outputs=outputs)

# Compile model
model_1.compile(optimizer=Adam(learning_rate=0.001), loss='categorical_crossentropy', metrics=['accuracy'])

# Deal with class imbalances
class_weights = compute_class_weight(class_weight='balanced', classes = np.unique(train_image_generator.classes), y= train_image_generator.classes)
class_weight_dict = dict(enumerate(class_weights))
history_1 = model_1.fit(
    train_image_generator,
    epochs=50,
    steps_per_epoch=327,
    validation_data=test_image_generator,
    class_weight=class_weight_dict
)
model_1.save('OCT_IV3.keras')
```

Python

InceptionV3 model

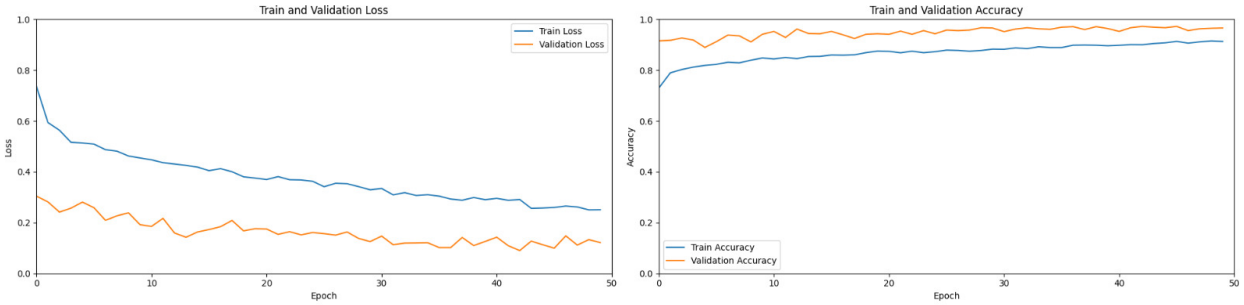


InceptionV3 training and validation data

```python
base_model_2 = VGG16(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet',
    pooling='max'
)
for layer in base_model_2.layers:
    layer.trainable = False

# Build model
inputs = base_model_2.input
x = BatchNormalization()(base_model_2.output)
x = Dense(1024, activation='relu')(x)
x = Dense(512, activation='relu')(x)
x = Dense(512, activation='relu')(x)
x = Dense(256, activation='relu')(x)
x = Flatten()(x)
outputs = Dense(4, activation='
model_2 = Model(inputs=inputs, outputs=outputs)

# Compile model
model_2.compile(optimizer=Adam(learning_rate=0.001), loss='categorical_crossentropy', metrics=['accuracy'])

# Deal with class imbalances
class_weights = compute_class_weight(class_weight='balanced', classes = np.unique(train_image_generator.classes), y= train_image_generator.classes)
class_weight_dict = dict(enumerate(class_weights))

checkpoint_callback = ModelCheckpoint(
    'best_model_VGG16.keras',
    save_best_only=True,
    mode='min'
)
history_2 = model_2.fit(
    train_image_generator,
    epochs=50,
    steps_per_epoch=327,
    validation_data=test_image_generator,
    callbacks=[checkpoint_callback],
    class_weight=class_weight_dict
)
model_2.save('OCT_VGG16.keras')
```

(function) outputs: Any

VGG16 Model



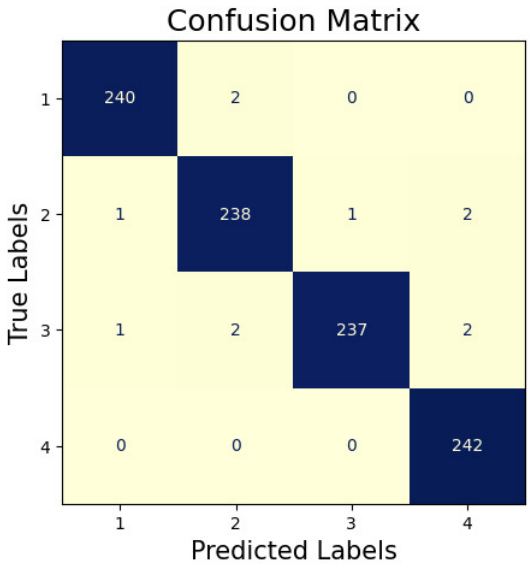
VGG16 training and validation data

```python
# Load test dataset with normalization
test_dataset = tf.keras.preprocessing.image_dataset_from_directory(
    test_path,
    image_size=img_size,
    batch_size=batch_size,
    seed=42
).map(preprocess_image)  # Apply hh using map function

# List of your models
model_paths = ['OCT_IRV2.keras', 'OCT_IV3.keras', 'best_model_VGG16.keras']  # Replace with your actual file paths
models = [tf.keras.models.load_model(model_path) for model_path in model_paths]

# Initialize lists to collect true labels and predictions
labels = []
ensemble_predictions = []

# Loop over the test dataset
for x, y in test_dataset:
    labels.append(list(y.numpy().astype("uint8")))

    # Collect predictions from all models
    batch_predictions = []
    for model in models:
        preds = model.predict(x)
        batch_predictions.append(np.argmax(preds, axis=1))

    # Stack predictions and perform majority voting
    batch_predictions = np.array(batch_predictions)  # Shape (num_models, batch_size)
    ensemble_pred_classes = stats.mode(batch_predictions, axis=0)[0].flatten()
    ensemble_predictions.append(ensemble_pred_classes)

# Flatten lists of labels and predictions
predictions = list(itertools.chain.from_iterable(ensemble_predictions))
labels = list(itertools.chain.from_iterable(labels))

# Calculate and print metrics
print("Test Accuracy    : {:.2f} %".format(accuracy_score(labels, predictions) * 100))
print("Precision Score : {:.2f} %".format(precision_score(labels, predictions, average='micro') * 100))
print("Recall Score     : {:.2f} %".format(recall_score(labels, predictions, average='micro') * 100))
print("F1 score         : {:.2f}".format(f1_score(labels, predictions, average='micro')))

# Optional: Confusion matrix
conf_matrix = confusion_matrix(labels, predictions)
print('Confusion Matrix:\n', conf_matrix)
print('Classification Report:\n', classification_report(labels, predictions))
```

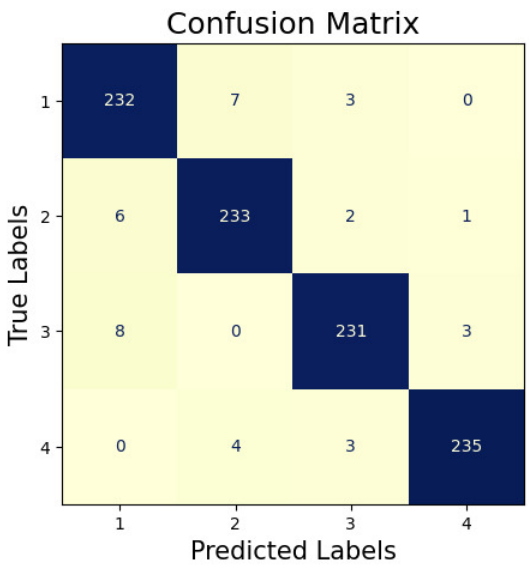
Ensemble training

```
Test Accuracy    : 98.86 %
Precision Score : 98.86 %
Recall Score     : 98.86 %
F1 score         : 0.99
Confusion Matrix:
 [[240   2   0   0]
 [  1 238   1   2]
 [  1   2 237   2]
 [  0   0   0 242]]
Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       242
           1       0.98      0.98      0.98       242
           2       1.00      0.98      0.99       242
           3       0.98      1.00      0.99       242

    accuracy                           0.99       968
   macro avg       0.99      0.99      0.99       968
weighted avg       0.99      0.99      0.99       968
```

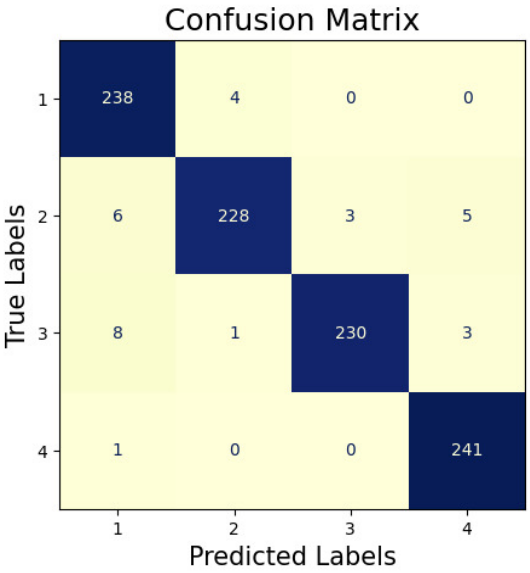
Ensemble results



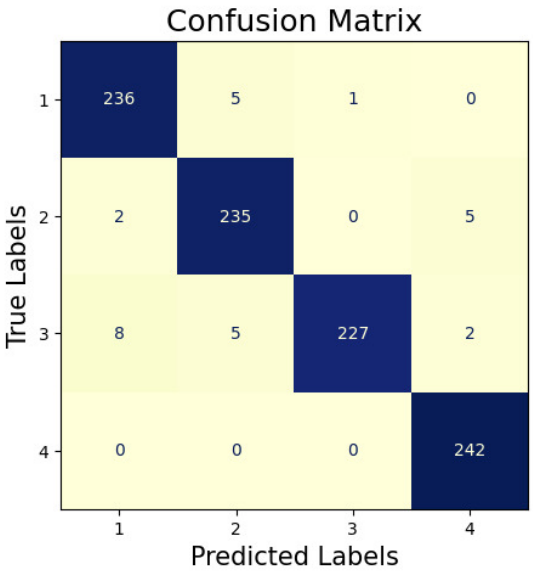
Ensemble Confusion Matrix



Inception ResNet Confusion Matrix



InceptionV3 Confusion Matrix



VGG16 Confusion Matrix